Facultad de Ciencias - UNAM
Lógica Computacional 2026-2
Práctica 4: Algoritmo DPLL

Favio Ezequiel Miranda Perea Patricio Ordoñez Blanco Eduardo Vargas Pérez

7 de Abril de 2026
Fecha de entrega: 23 de Abril de 2026

1. Introducción

A lo largo del curso se ha abordado la **satisfacibilidad** booleana utilizando diferentes técnicas, algoritmos y teoremas.

En este punto se puede hacer mención del *Teorema de Cook*, pues es de gran importancia para las ciencias de la computación en general. Con este teorema se introduce el concepto de NP-completud y se establece que el problema de satisfacibilidad booleana es un problema NP-completo; de hecho, el primero en ser demostrado que pertenece a esta clase de complejidad. Para mayor detalle y una breve introducción a este teorema, pueden revisar [1].

El problema SAT se puede definir como un problema de decisión que consiste en lo siguiente:

- **Ejemplar:** Un conjunto de cláusulas $C_1, C_2, ..., C_k$ en las que se usan en total n variables.
- **Pregunta:** ¿Existe un estado de las n variables tal que para toda $i \in \{1, ..., k\}$ se tiene $I(C_i) = 1$?

Al ser un problema NP-completo, no existe un algoritmo determinista que, dado un ejemplar del problema, pueda determinar si dicho ejemplar es satisfacible en tiempo polinomial. Sin embargo, a lo largo de los años se han desarrollado sistemas capaces de "resolver" ejemplares del problema SAT de manera relativamente eficiente. Por ejemplo, el algoritmo de saturación que se implementó en la práctica pasada (recuerden que resolución binaria es no determinista).

Si bien el algoritmo de saturación implementado nos ayuda a determinar la satisfacibilidad de una fórmula, este no da un modelo que satisfaga la fórmula en cuestión. Por otro lado, el algoritmo DPLL se basa en el uso de backtracking para tratar de construir un modelo que satisfaga a la fórmula.

Con el paso del tiempo, el algoritmo ha sido modificado con el fin de hacerlo cada vez más eficiente; sin embargo, en esta práctica se revisa una versión simplificada para propósitos didácticos.

2. Objetivo

El objetivo principal de esta práctica es que, por medio de la implementación del algoritmo DPLL (Davis-Putnam-Logemann-Loveland), refuercen los conocimientos teóricos sobre el problema de satisfacibilidad booleana y comprendan más a fondo la aplicación de las reglas en este algoritmo.

3. Desarrollo de la práctica

Primero, es importante mencionar que se estará trabajando con los mismos tipos de datos que se usaron para la práctica anterior. Es decir, los tipos de datos *Prop*, *Literal* y *Clausula*

3.1. Implementación del algoritmo DPLL (Parte 1)

El algoritmo se modela como un sistema de transiciones en donde $M \models F$ representa la búsqueda de un modelo M para F.

Para esta práctica, se define un `Estado del Algoritmo` como una pareja en donde la primera entrada corresponde a una `Interpretacion` de las variables, mientras que la segunda entrada corresponde a una `Fórmula` que será representada mediante una lista de cláusulas.

En código es lo siguiente:

```
type Interpretacion = [( String , Bool ) ]
type Estado = ( Interpretacion , [Clausula])
```

Por otro lado, las transiciones entre estados, que se modelan como $M \models_? F \quad \triangleright \quad M' \models_? F'$, se representan mediante la aplicación de reglas. Para ello, se deben desarrollar las reglas que componen el algoritmo.

Ejercicios

- *Conflicto* (`conflict :: Estado -> Bool`)

  Determina si la cláusula vacía, representada como una lista vacía, forma parte del conjunto de cláusulas. Puesto que no existe modelo que satisfaga a la cláusula vacía, la búsqueda del modelo falla. **[1 punto]**
  Ejemplos

  ```
  ghci > conflict ([] , [[p,q] , [r,s] , [] , [p,r,t]]) ▷ True
  ghci > conflict ([] , [[p,q] , [r,s] , [p,r,t]]) ▷ False
  ```

- *Éxito* (`success :: Estado -> Bool`)

  Determina si la búsqueda del modelo ha sido exitosa, esto sucede cuando el conjunto de cláusulas es vacío. **[1 punto]**
  Ejemplos

  ```
  ghci > success ([( "p", True ) , ("q", False ) ] , []) ▷ True
  ghci > success ([( "p", True ) , ("q", False ) ] , [[p,r]]) ▷ False
  ```

- *Cláusula unitaria* (`unit :: Estado -> Estado`)

  Si ℓ es una literal que pertenece a una cláusula unitaria, entonces basta con agregar ℓ al modelo y seguir con la búsqueda. Es importante notar que si $\ell^c \in M$ esta regla no puede aplicarse, de lo contrario se necesitaría que tanto ℓ como ℓ^c sean verdaderas, lo que conduce a una contradicción.
  [1 punto]
  Nota: Sólo se debe aplicar esta regla sobre la primer cláusula unitaria encontrada.

  Ejemplos

```
ghci > unit ([] , [[p,q] , [r,s] , [t]]) ▷
    ([( "t", True ) ] , [[p,q] , [r,s]])
ghci > unit ([] , [[p,q] , [r,s] , [Not t]]) ▷
    ([( "t", False ) ] , [[p,q] , [r,s]])
```

- *Eliminación* (`elim :: Estado -> Estado`)

  Si ℓ es una literal que pertenece al modelo en construcción M y se tiene alguna cláusula $\ell \vee C$, entonces, dado que ℓ es verdadera, $\ell \vee C$ también lo es. Por lo tanto, se elimina la cláusula $\ell \vee C$ del conjunto de cláusulas. **[1 punto]**
  Nota: Esta regla se aplica para cada una de las literales en la interpretación.

  Ejemplos

```
ghci > elim ([( "p", True ) ] , [[p,q] , [r,s] , [p,r,t]]) ▷
    ([( "p", True ) ] , [[r,s]])
ghci > elim ([( "p", False ) ] , [[Not p,q] , [r,s] , [Not p,r,t]]) ▷
    ([( "p", False ) ] , [[r,s]])
```

- *Reducción:* (`red :: Estado -> Estado`)

  Si ℓ es una literal que pertenece al modelo en construcción M y se tiene la cláusula $\ell^c \vee C$, entonces, dado que ℓ es verdadera, ℓ^c es falsa y sólo es de interés saber si C es satisfacible.
  [1 punto]
  Nota: Esta regla se aplica para cada una de las literales en la interpretación
  Ejemplos

```
ghci > red ([( "p", True ) ] , [[Not p,q] , [r,s] , [p,r,t]]) ▷
    ([( "p", True ) ] , [[q] , [r,s] , [p,r,t]])
ghci > red ([( "p", False ) ] , [[Not p,q] , [r,s] , [p,r,t]]) ▷
    ([( "p", False ) ] , [[Not p,q] , [r,s] , [r,t]])
```

- *Separación* (`sep :: Literal -> Estado -> (Estado, Estado)`)

  Dada una literal ℓ se procede a buscar que M, ℓ sea modelo de F, o que M, ℓ^c lo sea.
  [1 punto]

  Ejemplos

```
ghci > sep p ([] , [[ p , q ]]) ▷
(([( "p", True ) ] , [[p,q]]) , ([( "p", False ) ] , [[p,q]]) )
ghci > sep (Not p) ([] , [[ p , q ]]) ▷
(([( "p", True ) ] , [[p,q]]) , ([( "p", False ) ] , [[p,q]]) )
```

  Obsérvese como en esta regla se generan dos caminos a explorar, uno con M, ℓ y otro con M, ℓ^c . De modo que si el primer camino falla en la búsqueda de un modelo, se procede a realizar la búsqueda en el segundo camino (paso bactrack). Si éste último también falla, entonces no hay modelos.

3.2. Árboles DPLL

Para poder implementar el algoritmo DPLL de manera sencilla y sin tener problemas de tipos debido a la regla de separación, se hará uso de un tipo de dato llamado *ArbolDPLL*. Estos árboles proporcionarán soporte para poder manejar el caso de la regla de separación y permitirán analizar de manera clara el *Backtracking* utilizado en el algoritmo. El tipo será el siguiente:

```
data ArbolDPLL = Node Estado ArbolDPLL | Branch Estado ArbolDPLL ArbolDPLL | Void
```

Este tipo de dato es obligatorio que se utilice en la implementación del algoritmo, junto con las reglas anteriores.

Nota curiosa: Si lo analizan con detenimiento, se trata de una implementación en particular de los árboles vistos en la primer práctica.

3.3. Implementación de algoritmo DPLL (Parte 2)

Al ser DPLL un algoritmo no determinista, se debe definir una *heurística* para decidir sobre qué literal se aplicará la regla de separación (pues se puede aplicar sobre más de una literal). Para esto, se usará la siguiente función:

```
heuristicsLiteral :: [Clausula] -> Literal
```

La cual, dada una lista de cláusulas (fórmula en forma normal conjuntiva), regresa la literal que más apariciones tiene en la fórmula. **[1 punto]**

Así, usando los ejercicios definidos previamente, la función principal sería:

```
dpll :: [Clausula] -> Interpretacion
```

Esta función recibe una fórmula proposicional en forma normal clausular (representada por una lista de cláusulas) y devuelve una interpretación que satisfaga a la fórmula tras la ejecución del algoritmo DPLL iniciando con el estado $\emptyset \models_? F$. En caso de que la fórmula sea insatisfacible, la función deberá devolver una lista vacía. **[3 puntos]**

[Hasta 1 punto extra] Implementar la función

```
dpll2 :: Prop -> Interpretacion
```

La cual recibe una fórmula de la lógica proposicional cualquiera y devuelve una interpretación que satisfaga a dicha fórmula. En caso de que la fórmula sea insatisfacible, se debe regresar nuevamente la lista vacía.
Hint: Si quieren realizar este ejercicio, necesitan tener funcionando correctamente varias funciones de la práctica 3.

¡Buena suerte a todos! ☺☺☺

Nota: Para mayor referencia teórica sobre los conceptos de esta práctica, pueden revisar la nota 4 del curso.

Referencias

[1] Jorge L. Ortega Arjona. Breves notas sobre complejidad, Noviembre 2007. Vínculos Matemáticos, Vol. 82, Departamento de Matemáticas, Facultad de Ciencias, UNAM. URL: `http://mathmoo.unam.mx/jloa/publicaciones/complejidad.pdf`.